

Integración de Arquitectura de Microservicios

DESARROLLO FULLSTACK III_001V

INTEGRANTES: AARON LORCA ° CESAR ESTAY ° ELIAN BARRA

PROFESOR: CLAUDIO ROJAS

Contenido

1. Contexto	2
2. Arquitectura del Sistema	2
2.1 C1 – Diagrama de Contexto	3
2.2 C2 – Diagrama de Contenedores	4
2.3 Resumen de contenedores:	4
2.4 Componentes de Software:.....	5
3. Patrones de Diseños Aplicados.....	6
4. Modelo de Datos	8
4.1 user_credentials (ms-auth)	8
4.2 users (ms-user)	9
4.3 appointments (ms-appointment).....	9
4.4 waitlist_entries (ms-waitlist)	9
5.1 Flyway con ddl-auto: none	10
5.2 BFF como orquestador de lógica de negocio	10
5.3 NeonDB Serverless vs BD local en Kubernetes	10

1. Contexto

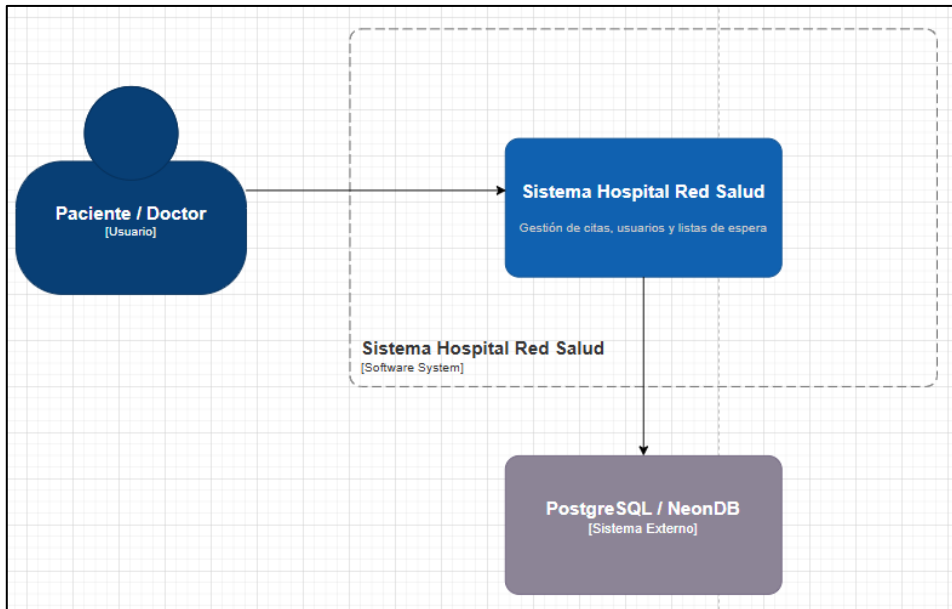
El Servicio Público de Salud RedNorte gestiona una red de establecimientos que atienden diariamente a miles de pacientes en distintas especialidades. El principal desafío es la gestión eficiente de listas de espera: cancelaciones sin reasignación, pérdida de horas médicas y escasa visibilidad para los pacientes. Para resolverlo, se desarrolló una plataforma digital con tres módulos: gestión integrada de listas de espera, reasignación automática de citas canceladas, y un portal de información para pacientes.

2. Arquitectura del Sistema

El sistema de gestión hospitalaria está construido sobre una arquitectura de microservicios desplegada en Kubernetes. Cada servicio autenticación, usuarios, citas y lista de espera es gestionado por un microservicio independiente con su propia base de datos en NeonDB (PostgreSQL 17 Serverless). El frontend en Next.js 16 se comunica exclusivamente a través de un API Gateway (Spring Cloud Gateway), que enruta las peticiones hacia un BFF (Backend for Frontend) responsable de orquestar los microservicios y ejecutar la lógica transversal del sistema, como la reasignación automática de citas. La autenticación es stateless basada en JWT firmados con RSA, validados por cada microservicio de forma independiente. Todo el sistema corre en el namespace hospital de Kubernetes con escalado automático habilitado en el servicio de mayor demanda.

2.1 C1 – Diagrama de Contexto

El nivel C1 muestra el sistema en su entorno y define quiénes son los actores externos que interactúan con él.

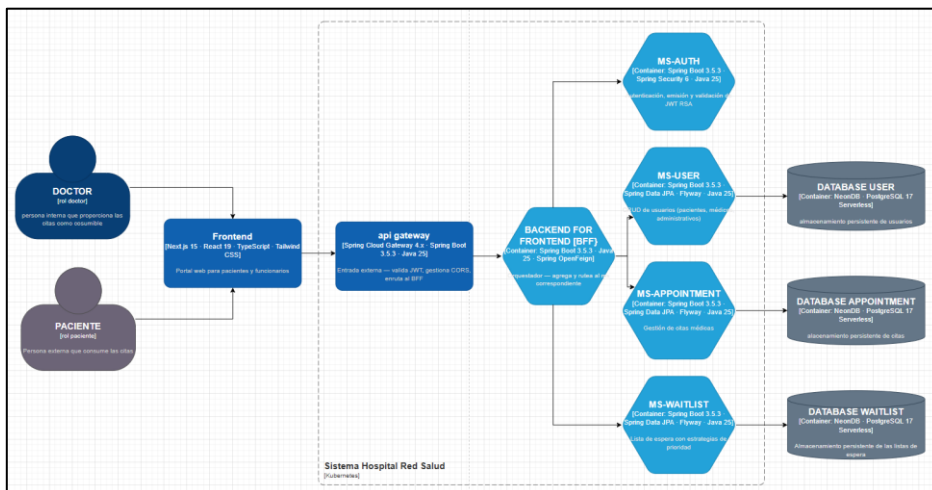


Los actores que interactúan con el sistema son:

- **Paciente:** Usuario externo que reserva, cancela y consulta citas médicas a través del Portal del Paciente (localhost:3001).
- **Doctor/Médico:** Personal clínico que cancela citas y visualiza su agenda desde el Portal del funcionario.
- **Administrador:** Gestor del sistema con acceso al dashboard administrativo para crear y gestionar usuarios.
- **NeonDB:** Base de datos externa cloud serverless PostgreSQL 17, gestionada por NeonDB, utilizada como almacenamiento persistente de todos los microservicios.

2.2 C2 – Diagrama de Contenedores

El nivel C2 descompone el sistema en sus contenedores de software principales, mostrando tecnologías, como interactúan entre ellos.



2.3 Resumen de contenedores:

Contenedor	Tecnología	Puerto	Responsabilidad
Frontend	Next.js 15 · React 19 · TypeScript	:3001	Portal Paciente y Funcionario.
API Gateway	Spring Cloud Gateway 4.x / SB 3.5.3	:8091	Punto de entrada único. JWT, CORS, rate limiting.
BFF	Spring Boot 3.5.3 · Java 25	:8090	Orquesta los 4 microservicios. ClusterIP (interno).
ms-auth	Spring Boot 3.5.3 · Spring Security 6	:8080	Autenticación BCrypt, emisión JWT RSA 2048-bit.
ms-user	Spring Boot 3.5.3 · Spring Data JPA	:8081	CRUD usuarios con Soft Delete.
ms-appointment	Spring Boot 3.5.3 · Spring Data JPA	:8082	Gestión de citas. HPA 2 réplicas.
ms-waitlist	Spring Boot 3.5.3 · Spring Data JPA	:8083	Cola de espera priorizada por especialidad.
NeonDB	PostgreSQL 17 Serverless	cloud	BD independiente por microservicio.

2.4 Componentes de Software:

Frontend

Aplicación web en Next.js 16 con React y TypeScript. Provee dos portales: uno para pacientes (reserva y cancelación de citas) y otro para funcionarios (médicos y administradores). Se comunica únicamente con el API Gateway mediante HTTP/REST con JWT en el header Authorization.

APIGateway

Único punto de entrada externo al sistema. Valida el JWT en cada request antes de enrutar hacia el BFF. Las rutas `/api/auth/**` son públicas (login y registro); todas las demás requieren token válido. Gestiona CORS centralizadamente.

BFF

Orquesta las llamadas a los microservicios internos y contiene la lógica transversal del sistema. Su componente más crítico es el `ReassignmentService`, que coordina el flujo cuando un médico cancela una cita: libera el slot, consulta la lista de espera y crea automáticamente una nueva cita para el siguiente paciente.

ms-auth

Gestiona la autenticación. Almacena credenciales con hash BCrypt, firma JWT con clave privada RSA 2048-bit y expone el endpoint `/.well-known/jwks.json` para que los demás servicios validen tokens de forma autónoma.

ms-user

CRUD de usuarios con tres roles: ADMIN, DOCTOR y PACIENTE. Implementa Soft Delete (campo `is_active`) para preservar el historial. Gestiona el campo `specialty` para médicos.

ms-appointment

Gestiona el ciclo de vida de las citas. Valida que no existan solapamientos de horario para un mismo médico. Es el servicio de mayor carga, por lo que tiene HPA configurado (mínimo 2, máximo 6 réplicas).

ms-waitlist

Gestiona la lista de espera con priorización dinámica: `vitalRisk` primero, luego `CRITICO` > `URGENTE` > `NORMAL`, luego por antigüedad. Cuando un médico cancela, el BFF consulta este servicio para obtener el siguiente paciente.

3. Patrones de Diseños Aplicados

El sistema aplica de manera consistente un conjunto de patrones de diseño arquitectónicos y de implementación que garantizan escalabilidad, mantenibilidad e independencia entre los componentes.

BFF (Backend for Frontend)

Un servicio Spring Boot dedicado exclusivamente a servir al frontend. Evita que el cliente web deba comunicarse con múltiples microservicios, simplifica la autenticación y centraliza la orquestación de flujos complejos como la reasignación automática de citas.

API Gateway (Edge Service)

Spring Cloud Gateway actúa como el único punto de entrada externo (:8091). Centraliza JWT validation mediante filtros de Spring Security, gestiona CORS, aplica RequestHeaderSize y enruta las peticiones al BFF. Ningún microservicio queda expuesto directamente al exterior.

Database per Service

Cada microservicio posee su propia base de datos independiente en NeonDB. No existen foreign key constraints cruzadas entre bases de datos. Las relaciones se mantienen por UUID de forma lógica. Esto garantiza despliegue autónomo y que un fallo en una BD no afecte a otros servicios.

Stateless JWT Authentication

Ningún microservicio guarda estado de sesión. ms-auth firma los JWT con clave privada RSA. Los demás servicios validan tokens descargando la clave pública desde el endpoint JWKS (/well-known/jwks.json). Política SessionCreationPolicy.STATELESS en todos los servicios.

Soft Delete

Los usuarios no se eliminan físicamente de la base de datos. En su lugar, el campo `is_active` se pone a `false`. Esto preserva la integridad referencial, el historial clínico y permite recuperar cuentas sin pérdida de datos.

Flyway Migrations

La flag `ddl-auto: none` impide que Hibernate toque el esquema de base de datos. Flyway gestiona el 100% del esquema mediante archivos SQL versionados (V1, V2, V3...). Cada microservicio arranca aplicando automáticamente las migraciones pendientes. Esto garantiza reproducibilidad total del entorno.

Facade

Aplicado en `AuthService` de `ms-auth` (documentado explícitamente en el código). Oculta la complejidad de BCrypt, JWT asimétrico con RSA y el repositorio detrás de métodos simples (`login`, `registerCredential`, `validateToken`) que el Controller consume sin conocer los detalles internos.

Repository

Todos los microservicios usan Spring Data JPA con interfaces que extienden `JpaRepository`. La capa de acceso a datos queda completamente abstraída: el Service no conoce SQL, solo trabaja con métodos del repositorio.

DTO (Data Transfer Object)

Cada microservicio define sus propios DTOs en `dto/request/` y `dto/response/`, separando el contrato de la API de la entidad JPA. Los nombres siguen la convención `XxxRequestDTO` / `XxxResponseDTO`.

Builder

Utilizado extensivamente con la anotación `@Builder` de Lombok en entidades y DTOs. Permite construcción legible y segura de objetos complejos sin constructores con múltiples parámetros.

Strategy

En WaitlistService el ordenamiento de la cola está encapsulado en un Comparator<WaitlistEntry> estático (QUEUE_ORDER) que implementa la estrategia de priorización: vitalRisk=true primero, luego CRITICO > URGENTE > NORMAL, luego por antigüedad (requeuedAt ASC). Cambiar la estrategia de priorización implica modificar solo ese comparador.

Orchestrator

ReasignacionService en el BFF implementa el patrón de orquestador: coordina múltiples microservicios (ms-appointment, ms-waitlist) en una secuencia definida de pasos para completar un flujo de negocio complejo, sin que los microservicios individuales se conozcan entre sí.

4. Modelo de Datos

El sistema aplica el patrón Database per Service: cada microservicio gestiona su propio esquema de base de datos de forma completamente independiente en NeonDB (PostgreSQL 17 Serverless). No existen JOINS ni foreign key constraints cruzadas entre bases de datos. Las relaciones lógicas se mantienen mediante UUIDs.

4.1 user_credentials (ms-auth)

Almacena las credenciales de autenticación. Separada de los datos de perfil (ms-user) para desacoplar autenticación de identidad.

- id (UUID, PK): identificador único autogenerado con gen_random_uuid()
- email (VARCHAR 255, UNIQUE): identificador de login
- password (VARCHAR 255): hash BCrypt con salt factor 10
- role (ENUM): ADMIN | DOCTOR | PACIENTE
- user_id (UUID, FK lógico): referencia al perfil en ms-user
- is_active (BOOLEAN): Soft Delete

4.2 users (ms-user)

Almacena el perfil completo del usuario: datos personales, tipo de documento, especialidad médica.

- id (UUID, PK): mismo UUID referenciado en user_credentials.user_id
- first_name, last_name, email, phone: datos personales
- document_type, document_number: RUT u otro documento
- role (ENUM): ADMIN | DOCTOR | PACIENTE
- specialty (VARCHAR 50): especialidad médica (nullable para pacientes)
- is_active (BOOLEAN): Soft Delete. created_at, updated_at.

4.3 appointments (ms-appointment)

Almacena las citas médicas. El campo status implementa la máquina de estados del ciclo de vida de una cita.

- id (UUID, PK)
- patient_id, doctor_id (UUID, FK lógico): referencias a usuarios en ms-user
- scheduled_at (TIMESTAMP): fecha y hora de la cita
- specialty (VARCHAR 50): especialidad de la cita
- duration_minutes (INT, default 30): duración de la cita en minutos
- appointment_type (VARCHAR 20, default 'CONSULTA'): tipo de atención
- cancelled_by (VARCHAR 10, nullable): DOCTOR o PATIENT — determina el flujo de reasignación
- active (BOOLEAN, default true): Soft Delete
- status (ENUM): PENDING | CONFIRMED | CANCELLED | COMPLETED | NO_SHOW
- notes (TEXT): observaciones clínicas

4.4 waitlist_entries (ms-waitlist)

Cola de espera priorizada por especialidad. La prioridad se determina por requested_at (FIFO). Al reencolar a un paciente que cancela, se actualiza requeued_at para que pierda prioridad.

- id (UUID, PK)
- patient_id (UUID, FK lógico): paciente en espera
- specialty (VARCHAR 50): especialidad solicitada
- status (ENUM): WAITING | NOTIFIED | ASSIGNED | OFFERED | CANCELLED
- requested_at (TIMESTAMP): timestamp original de solicitud (determina prioridad FIFO)
- requeued_at (TIMESTAMP): se actualiza al reencolarse, reduciendo prioridad

- offered_appointment_id (UUID): referencia a la cita ofrecida (cuando status=OFFERED)5. Decisiones Críticas

5.1 Flyway con ddl-auto: none

Se desactivó completamente la gestión automática de esquemas de Hibernate (ddl-auto: none). Flyway gestiona el 100% del esquema mediante archivos SQL versionados y auditados. Los archivos de migración forman parte del repositorio Git.

Alternativa descartada	Justificación de la elección
ddl-auto: create-drop (destruye datos en cada reinicio) o ddl-auto: update (cambios no rastreables, peligroso en producción).	Reproducibilidad total: cualquier entorno que arranque el microservicio obtiene exactamente el mismo esquema. Los cambios son rastreables en Git (quién cambió qué y cuándo).

5.2 BFF como orquestador de lógica de negocio

La lógica de reasignación automática de citas (cancelar > buscar paciente en waitlist > crear nueva cita) reside en el BFF, específicamente en ReassignmentService. El AppointmentService del BFF es solo un cliente REST sin lógica de negocio. Los microservicios individuales son servicios de datos puros (CRUD) y no se conocen entre sí.

Alternativa descartada	Justificación de la elección
(Kafka/RabbitMQ): mayor complejidad operacional sin beneficio real para la escala actual del sistema.	El BFF puede implementar flujos multi-paso con rollback explícito sin la complejidad de un bus de mensajes. Apropiado para la escala del sistema.

5.3 NeonDB Serverless vs BD local en Kubernetes

Se eligió NeonDB (PostgreSQL 17 Serverless) en lugar de desplegar instancias de PostgreSQL dentro del clúster de Kubernetes.

Alternativa descartada	Justificación de la elección
PostgreSQL en pods de Kubernetes: requiere gestión de volúmenes persistentes (PVC), backups manuales y mayor complejidad operacional.	NeonDB ofrece escalado automático, backups automáticos, SSL obligatorio y branching de BD para desarrollo. Reduce la carga operacional del equipo.